Lasso Regression

In [ ]:
```
Least Absolute Shrinkage and selection operat

Linear Regression triesz to find the best relationship between

X and Y

Y=m1x1+m2x2+m3X3+....+MnXn

Lasso adds  penalty to large coefficients

Features          Coefficient

Area              150
Bedrooms          25
Age               -8
Distance          -2
Garden             0
Balcony            0

After apply Lasso:

Features          Coefficient

Area              145
Bedrooms          21
Age               -6
Distance          -1
Garden             0
Balcony            0

Y=Price

Lasso can perform feature selection


LR  :-->RSS

to minimize prediction Error

Ridge:(L2 Regularization) -->coefficients become small
Penalty

Lasso (L1 Regularization) --->some coeffients can become exactly 0

After Lr training

X1=3
m1=4
Y=m1x1

Y=12

Ridge:

X1=3
m1= 2
Y=m1x1

Y=6
```

In [ ]:
```
Lambda 0    ---->Linear regression

Small lambda ---->small penalty

Large Lambda  --->Strong Penalty

Very Large --->  More coeffiecients become 0



predicted salary=m0+m1*Experience + m2*Education+m3*Age+m4*Location+m5*Certifications

After lasso

Experience   ---->4500
Education --->2500
Age   ---->0
Location --->1200
Certification  --->1800


200

040
```

In [2]:
```
import pandas as pd
import numpy as np


data={
    'Experience':[1,2,3,4,5,6,7,8,9,10],
    'Education':[1,1,2,2,2,3,3,3,4,4],
    'Age':[22,24,25,27,29,31,33,35,37,40],
    'Location':[2,3,4,2,4,5,4,5,5,4],
    'salary':[30000,35000,42000,48000,55000,62000,70000,76000,86000,95000]
}
```

In [6]:
```
df=pd.DataFrame(data)
```

In [8]:
```
df
```

Out[8]:

	Experience	Education	Age	Location	salary
0	1	1	22	2	30000
1	2	1	24	3	35000
2	3	2	25	4	42000
3	4	2	27	2	48000
4	5	2	29	4	55000
5	6	3	31	5	62000
6	7	3	33	4	70000
7	8	3	35	5	76000
8	9	4	37	5	86000
9	10	4	40	4	95000

In [10]:
```
Y=df[['salary']]
X=df.drop(labels=['salary'],axis=1)
```

In [12]:
```
Y
```

Out[12]:

	salary
0	30000
1	35000
2	42000
3	48000
4	55000
5	62000
6	70000
7	76000
8	86000
9	95000

In [14]:
```
X
```

Out[14]:

	Experience	Education	Age	Location
0	1	1	22	2
1	2	1	24	3
2	3	2	25	4
3	4	2	27	2
4	5	2	29	4
5	6	3	31	5
6	7	3	33	4
7	8	3	35	5
8	9	4	37	5
9	10	4	40	4

In [16]:
```
from sklearn.model_selection import train_test_split
xtrain,xtest,ytrain,ytest=train_test_split(X,Y,test_size=0.2,random_state=42)
```

Standardization

In [19]:
```
from sklearn.preprocessing import StandardScaler
sc=StandardScaler()

xtrain_scaled=sc.fit_transform(xtrain)
xtest_scaled=sc.fit_transform(xtest)
```

In [21]:
```
xtrain_scaled
```

Out[21]:
```
array([[ 0.18569534,  0.57735027,  0.13882344,  1.14707867],
       [-1.67125804, -1.73205081, -1.52705788, -1.60591014],
       [ 0.92847669,  0.57735027,  0.87921515,  1.14707867],
       [-0.92847669, -0.57735027, -0.97176411,  0.22941573],
       [ 1.67125804,  1.73205081,  1.80470477,  0.22941573],
       [-0.18569534, -0.57735027, -0.23137241,  0.22941573],
       [-0.55708601, -0.57735027, -0.60156826, -1.60591014],
       [ 0.55708601,  0.57735027,  0.50901929,  0.22941573]])
```

In [23]:
```
xtest_scaled
```

Out[23]:
```
array([[ 1.,  1.,  1.,  1.],
       [-1., -1., -1., -1.]])
```

Lasso Model

In [26]:
```
from sklearn.linear_model import Lasso
lasso=Lasso(alpha=0.1)
model=lasso.fit(xtrain_scaled,ytrain)
```

```
C:\Users\Lenovo\anaconda3\Lib\site-packages\sklearn\linear_model\_coordinate_descent.py:697: ConvergenceWarning: Objective did not converge. You might want to increase the number of iteration
s, check the scale of the features or consider increasing regularisation. Duality gap: 6.298e+05, tolerance: 2.978e+05
  model = cd_fast.enet_coordinate_descent(
```

In [28]:
```
y_pred=model.predict(xtest_scaled)
```

In [30]:
```
y_pred
```

Out[30]:
```
array([79016.36350329, 40483.63649671])
```

Evaluate

In [36]:
```
from sklearn.metrics import r2_score,mean_squared_error
print(r2_score(ytest,y_pred))
```

```
0.9393760492530965
```

In [38]:
```
mean_squared_error(ytest,y_pred)
```

Out[38]:
```
39420723.97317399
```

In [40]:
```
ytest
```

Out[40]:

	salary
8	86000
1	35000

In [42]:
```
y_pred
```

Out[42]:
```
array([79016.36350329, 40483.63649671])
```

In [48]:
```
coeffdf=pd.DataFrame({
    'Feature':X.columns,
    'Coefficient': model.coef_
})
```

In [50]:
```
coeffdf
```

Out[50]:

	Feature	Coefficient
0	Experience	-633.080215
1	Education	1183.632773
2	Age	18871.912453
3	Location	-156.101508

In [54]:
```
alpha_list=[0.001,0.01,0.1,1,10,100]
from warnings import filterwarnings
filterwarnings('ignore')
for i in alpha_list:
    model=Lasso(alpha=i)
    model.fit(xtrain_scaled,ytrain)

    print("Alpha: ",i)
    print("Coefficient: ",model.coef_)
```

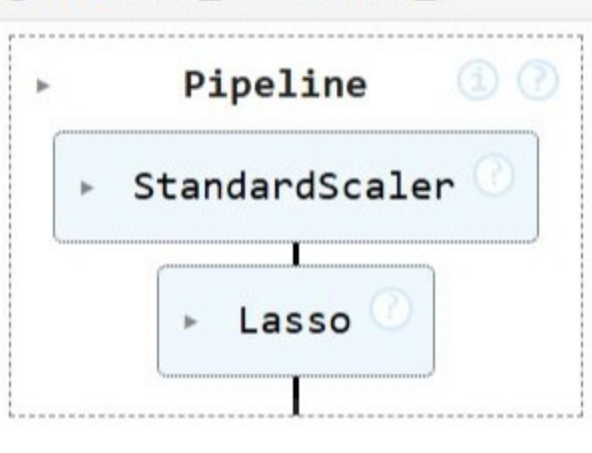
```
Alpha:  0.001
Coefficient:  [ -665.9795163   1185.5833081  18902.10697039  -154.74014753]
Alpha:  0.01
Coefficient:  [ -662.99320657  1185.40578596 18899.36655672  -154.8636248 ]
Alpha:  0.1
Coefficient:  [ -633.08021483  1183.6327728  18871.91245296  -156.10150765]
Alpha:  1
Coefficient:  [ -195.52286124  1172.0365222  18458.73617821  -177.1101146 ]
Alpha:  10
Coefficient:  [    0.          1140.93664821 18275.43327554  -161.53725972]
Alpha:  100
Coefficient:  [    0.           922.82004349 18297.32451766    -0.        ]
```

In [60]:
```
from sklearn.pipeline import Pipeline
from sklearn.model_selection import GridSearchCV

pipeline=Pipeline([
    ('scaler',StandardScaler()),
    ('lasso',Lasso())
])

params={
    'lasso__alpha':[0.001,0.01,0.1,1,10,100]
}

grid=GridSearchCV(
    pipeline,
    params,
    cv=5,
    scoring='r2'
)

grid.fit(xtrain,ytrain)

print("Best Alpha: ",grid.best_params_)

print("Best Score: ",grid.best_score_)
```

```
Best Alpha:  {'lasso__alpha': 0.001}
Best Score:  nan
```

In [62]:
```
grid.best_estimator_
```

Out[62]:



In [66]:
```
y_pred=grid.predict(xtest)
```

In [68]:
```
y_pred
```

Out[68]:
```
array([84376.80666212, 36801.56408138])
```

In [72]:
```
ytest['y_pred']=y_pred
```

In [74]:
```
ytest
```

Out[74]:

	salary	y_pred
8	86000	84376.806662
1	35000	36801.564081

In [ ]: